Blockchain : Experiment - 8

Aim:

Configuring a Cryptocurrency Wallet using Metamask and integrating it with a DApp.

Theory:

The Problem in a Pharmaceutical Supply Chain

A pharmaceutical supply chain involves several participants, including manufacturers, distributors, retailers, and customers. Each participant handles a different stage of a medicine's journey, making it important to maintain accurate and reliable records. Traditional systems often maintain separate databases, making it difficult to track the complete history of a drug. This can lead to counterfeit medicines, the sale of expired products, and delays in identifying the source of a problem. A blockchain-based system helps maintain a shared record of drug movements and improves traceability across the supply chain.

MetaMask and Decentralized Applications (DApps)

MetaMask is a cryptocurrency wallet that allows users to manage blockchain accounts, connect to blockchain networks, and authorize transactions. A decentralized application (DApp) provides a user interface through which users interact with smart contracts deployed on a blockchain. In this experiment, a web-based DApp is developed using HTML, CSS, and JavaScript, with MetaMask providing wallet connectivity and transaction authorization.

Integration Using Ethers.js

Ethers.js is a JavaScript library that enables a web application to communicate with the Ethereum blockchain and interact with deployed smart contracts. It uses the contract's address and Application Binary Interface (ABI) to access its functions. The DApp connects to MetaMask through the browser's Ethereum provider, allowing users to retrieve blockchain data and initiate transactions. MetaMask requests user approval before signing and submitting transactions, which require gas fees paid using the network's cryptocurrency.

Design of the DrugSupplyChain Smart Contract

The DrugSupplyChain contract is developed using Solidity to manage the movement of medicines through different supply-chain participants. It maintains drug records and enforces the correct sequence of operations.

- enum Stage: Defines the four stages of a drug: Manufactured, WithDistributor, WithRetailer, and Sold.
- struct Drug: Stores the drug ID, name, expiry date, manufacturer, distributor, retailer, customer, current stage, and last-updated timestamp.
- mapping(uint256 => Drug): Stores drug records on the blockchain using the drug ID as the key.
- Smart contract functions: manufactureDrug(), sendToDistributor(), sendToRetailer(), sellToCustomer(), and getDrug() allow the DApp to register drugs, update their custody, record sales, and retrieve their details.
- require() checks: Validate the caller's address, the current drug stage, and the expiry date before allowing relevant state changes. Invalid operations are reverted by the contract.

These rules ensure that only the assigned participant can perform the corresponding supply-chain operation and that the intended sequence of drug transfers is maintained.

Ethereum Sepolia Testnet

The Ethereum Sepolia testnet provides an environment for deploying and testing smart contracts without using real-value Ether. In this experiment, the DrugSupplyChain contract is deployed on Sepolia using Remix and MetaMask. Sepolia test ETH is used to pay transaction gas fees. The deployed contract is then connected to the DApp using its address and ABI, enabling users to interact with the contract and verify transaction records through a blockchain explorer such as Etherscan.

Working of the DApp

The DApp connects the user's MetaMask wallet to the Sepolia network and uses Ethers.js to communicate with the deployed smart contract. The user can register a drug, transfer it between supply-chain participants, and retrieve its current details through the web interface. Each state-changing operation requires wallet authorization and is recorded on the blockchain after successful execution. This integration demonstrates how MetaMask, a web-based interface, and Solidity smart contracts work together to support transparent and traceable pharmaceutical supply-chain management.

Smart Contract Code:

```
pragma solidity ^0.8.0;
contract DrugSupplyChain {
  enum Stage { Manufactured, WithDistributor,
WithRetailer, Sold }
  struct Drug {
    uint256 drugID;
    string name;
    uint256 expiryDate;
    address manufacturer;
    address distributor;
    address retailer;
    address customer;
    Stage stage;
    uint256 lastUpdated;
  }
  mapping(uint256 => Drug) public drugs;
  event DrugManufactured(uint256 drugID,
string name, address manufacturer);
  event SentToDistributor(uint256 drugID,
address distributor);
  event SentToRetailer(uint256 drugID, address
retailer);
  event SoldToCustomer(uint256 drugID,
address customer);
  function manufactureDrug(uint256 _drugID,
string memory _name, uint256 _expiryDate)
public {
    require(_drugID != 0, "Invalid drug ID");
    require(drugs[_drugID].drugID == 0, "Drug
already exists");
    drugs[_drugID] = Drug({
      drugID: _drugID,
      name: _name,
      expiryDate: _expiryDate,
      manufacturer: msg.sender,
      distributor: address(0),
      retailer: address(0),
      customer: address(0),
      stage: Stage.Manufactured,
      lastUpdated: block.timestamp
    });
    emit DrugManufactured(_drugID, _name,
msg.sender);
  }
  function sendToDistributor(uint256 _drugID,
address _distributor) public {
    Drug storage d = drugs[_drugID];
    require(d.drugID != 0, "Drug does not
exist");
    require(msg.sender == d.manufacturer,
"Only manufacturer allowed");
    require(d.stage == Stage.Manufactured,
"Wrong stage");
    d.distributor = _distributor;
    d.stage = Stage.WithDistributor;
    d.lastUpdated = block.timestamp;
    emit SentToDistributor(_drugID,
_distributor);
  }
  function sendToRetailer(uint256 _drugID,
address _retailer) public {
    Drug storage d = drugs[_drugID];
```

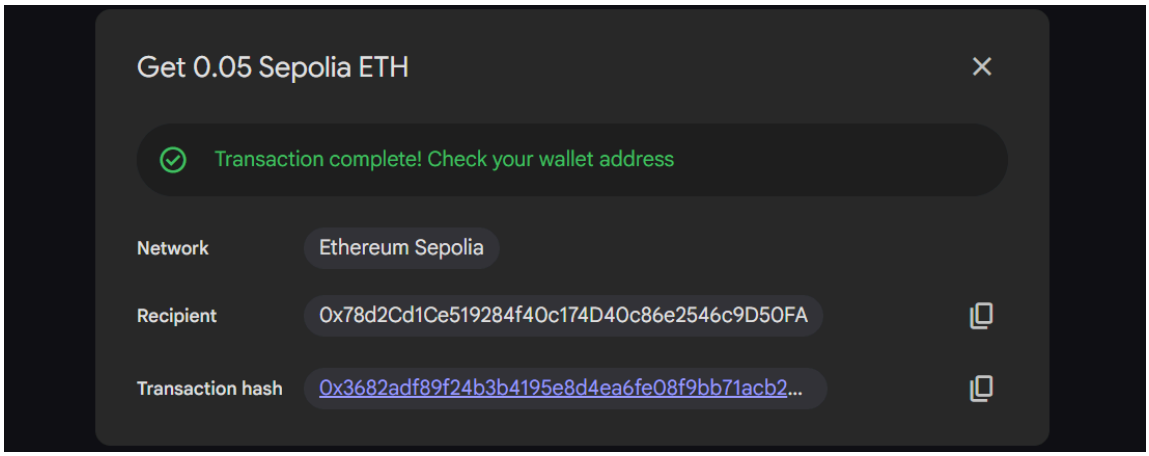
```
        require(d.drugID != 0, "Drug does not
exist");
        require(msg.sender == d.distributor, "Only
distributor allowed");
        require(d.stage == Stage.WithDistributor,
"Wrong stage");
        d.retailer = _retailer;
        d.stage = Stage.WithRetailer;
        d.lastUpdated = block.timestamp;
        emit SentToRetailer(_drugID, _retailer);
    }
    function sellToCustomer(uint256 _drugID,
address _customer) public {
        Drug storage d = drugs[_drugID];
        require(d.drugID != 0, "Drug does not
exist");
        require(msg.sender == d.retailer, "Only
retailer allowed");
        require(d.stage == Stage.WithRetailer,
"Wrong stage");
        require(block.timestamp < d.expiryDate,
"Drug expired");
        d.customer = _customer;
        d.stage = Stage.Sold;
        d.lastUpdated = block.timestamp;
        emit SoldToCustomer(_drugID, _customer);
    }
    function getDrug(uint256 _drugID) public
view returns (Drug memory) {
        require(drugs[_drugID].drugID != 0, "Drug
does not exist");
        return drugs[_drugID];
    }
}
```

DApp Code:

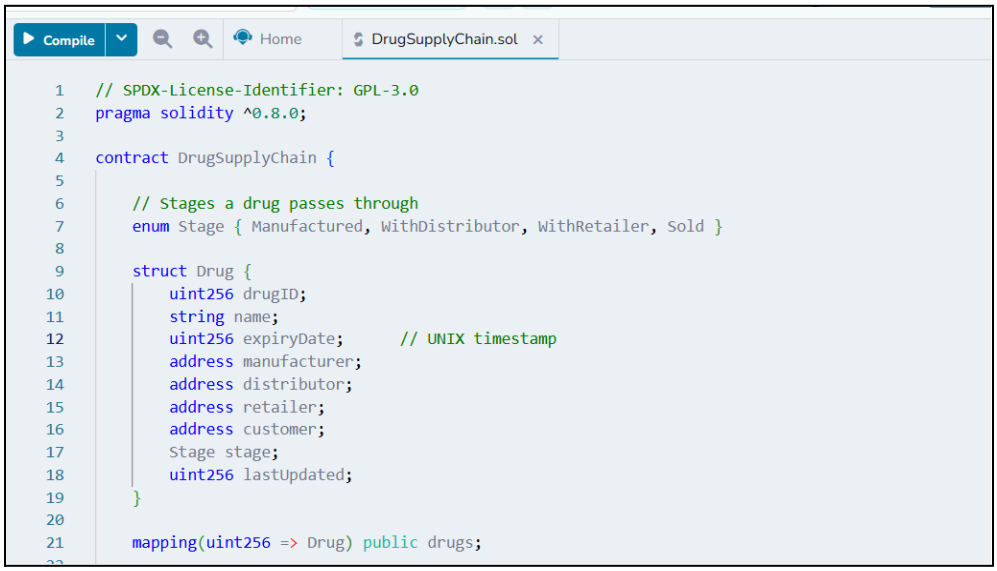
As the DApp code is too large, we have put it here:
[Blockchain/index.html at main · ShaktiMS/Blockchain]

Output:

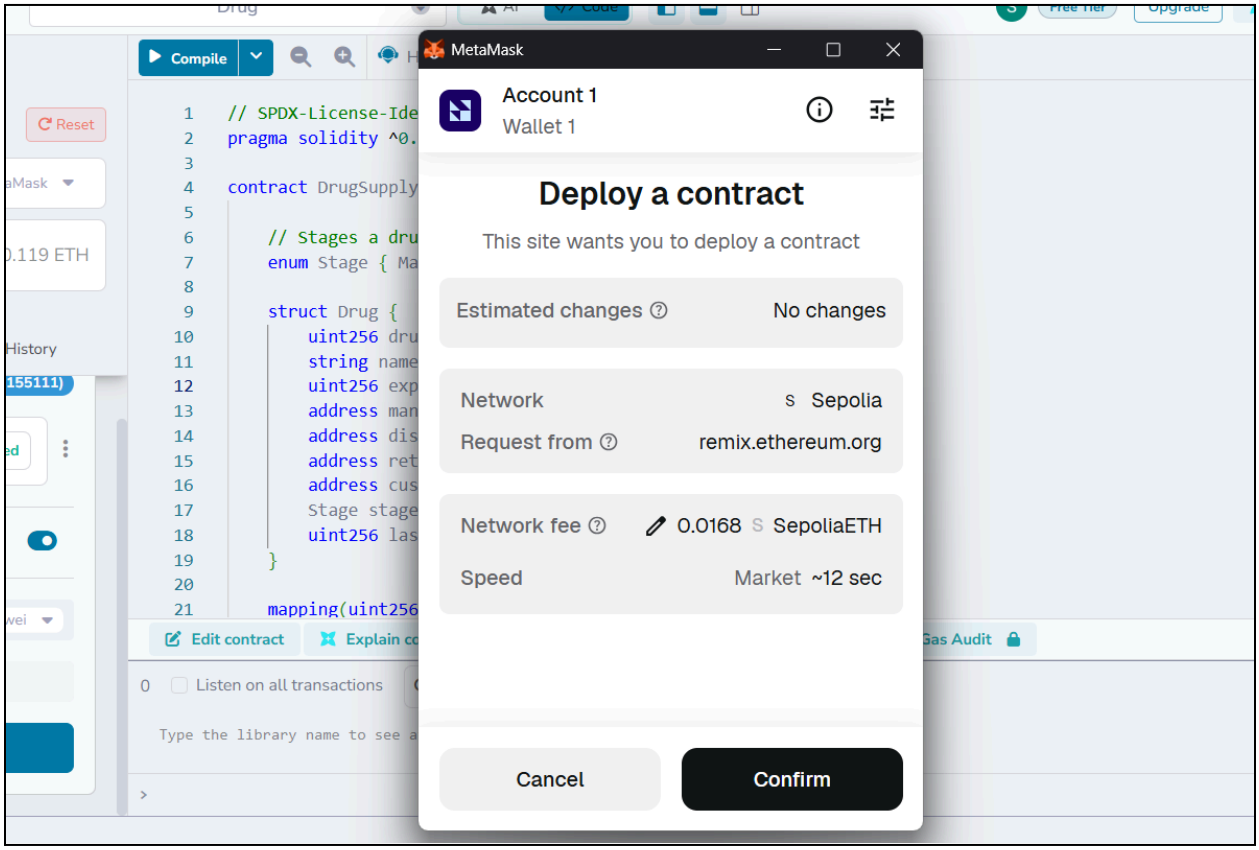
Get Sepolia Testnet Eth:



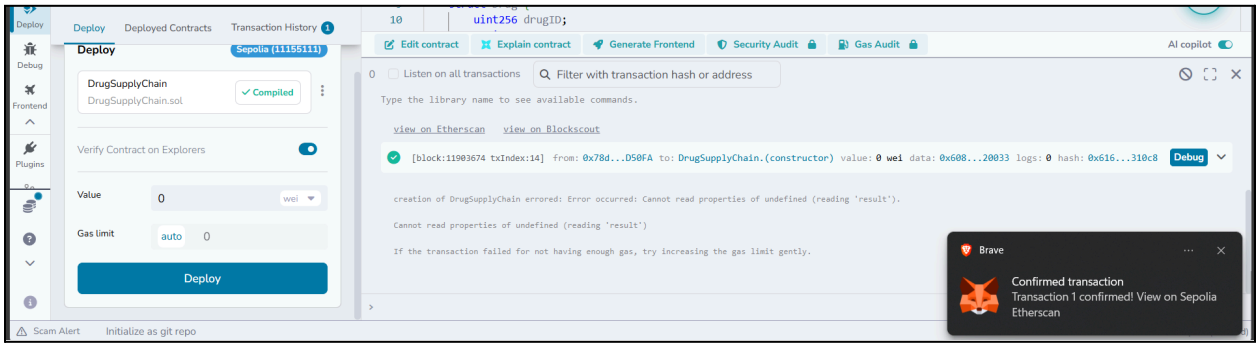
Create Smart Contract:



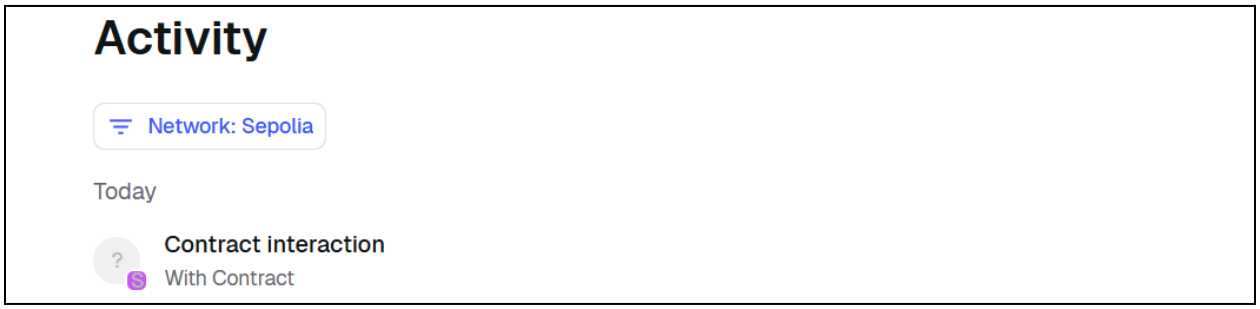
Deploy Smart Contract using Sepoli Testnet and Metamask:

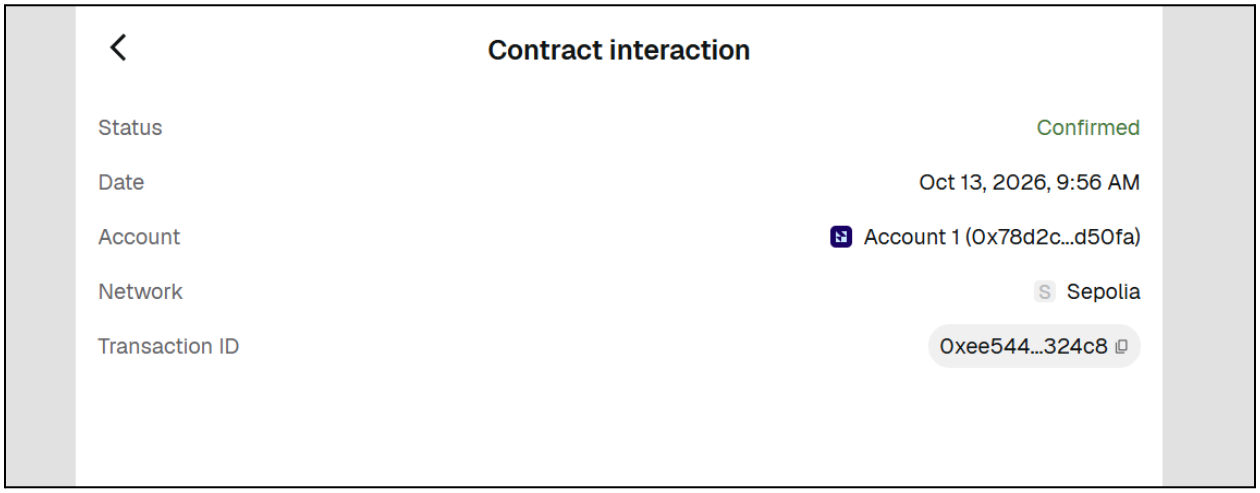


Deployed Smart Contract:

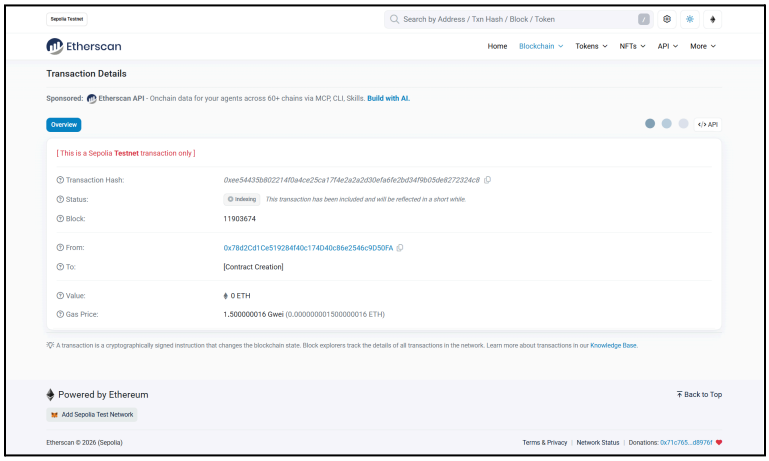


Smart Contract Transactions:

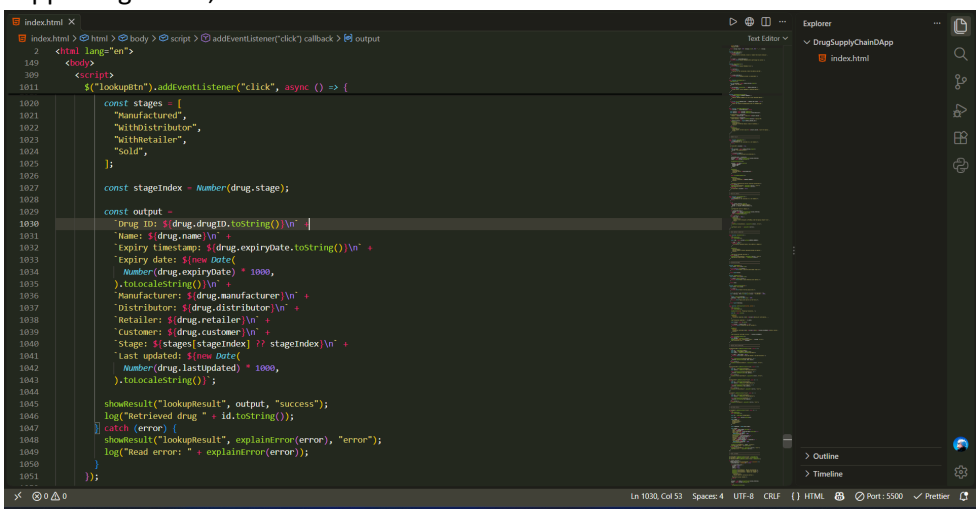




Check the transaction on Etherscan:



Create the DApp using HTML, CSS and JS:



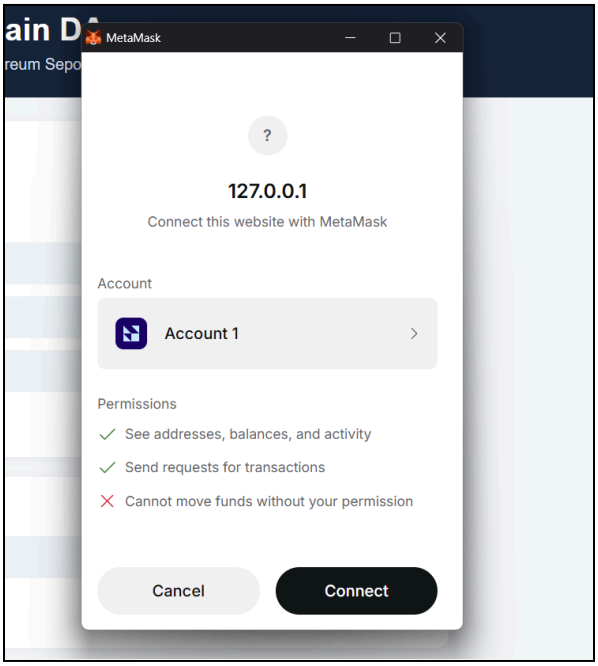
```
<html lang="en">
  <body>
    <script>
      ${"lookupBtn").addEventListener("click", async () => {
        const stages = [
          "Manufactured",
          "WithDistributor",
          "WithRetailer",
          "Sold",
        ];

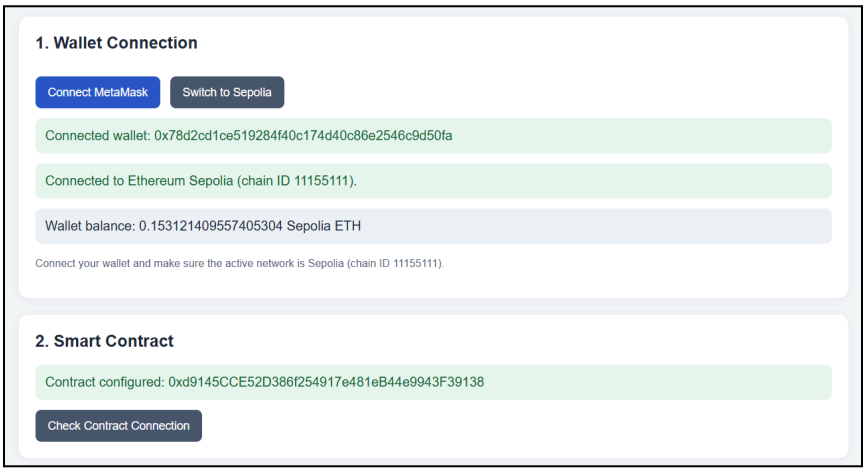
        const stageIndex = Number(drug.stage);

        const output =
          `Drug ID: ${drug.drugID.toString()}\n` +
          `Name: ${drug.name}\n` +
          `Expiry timestamp: ${drug.expiryDate.toString()}\n` +
          `Expiry date: ${new Date(
            Number(drug.expiryDate) * 1000,
          ).toLocaleString()}\n` +
          `Manufacturer: ${drug.manufacturer}\n` +
          `Distributor: ${drug.distributor}\n` +
          `Retailer: ${drug.retailer}\n` +
          `Customer: ${drug.customer}\n` +
          `Stage: ${stages[stageIndex] ?? stageIndex}\n` +
          `Last updated: ${new Date(
            Number(drug.lastUpdated) * 1000,
          ).toLocaleString()}`;

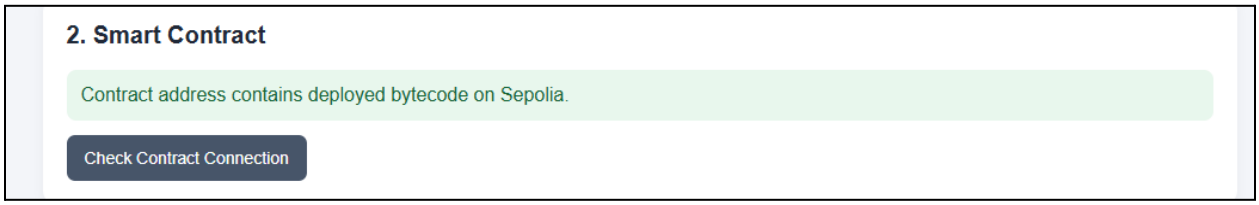
        showResult("lookupResult", output, "success");
        log("Retrieved drug " + id.toString());
      } catch (error) {
        showResult("lookupResult", explainError(error), "error");
        log("Read error: " + explainError(error));
      }
    });
```

Connect the DApp with MetaMask:

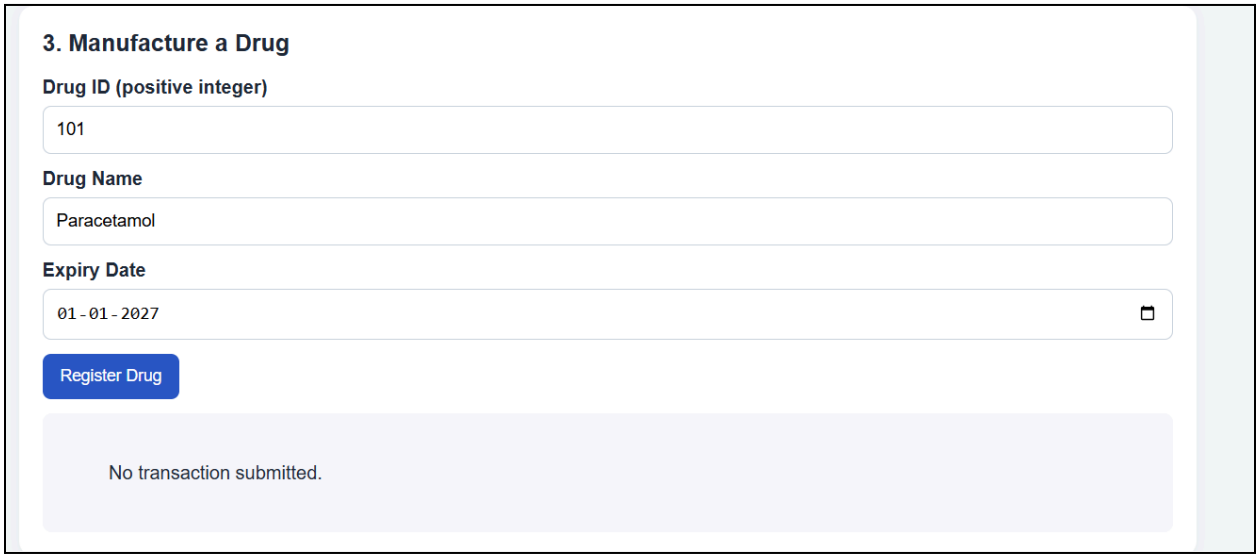


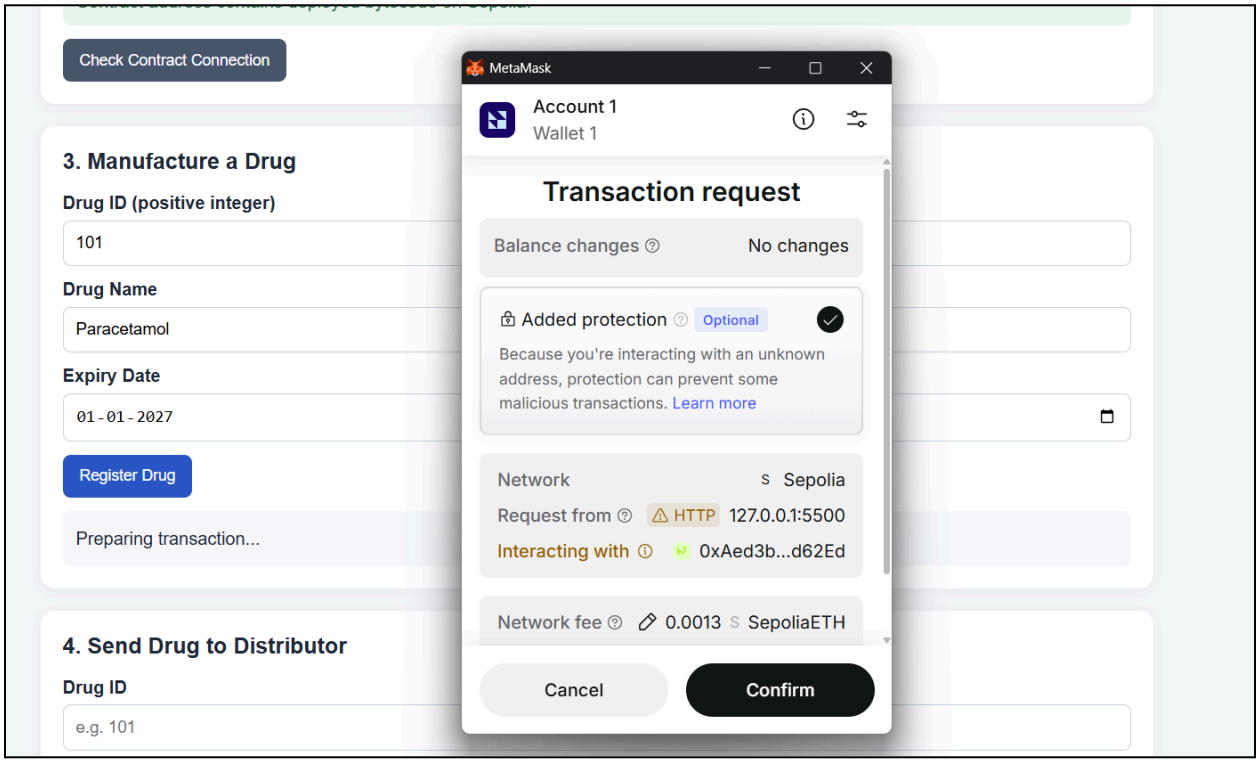


Check Contract Connection:



Testing the Dapp by registering a new drug:







Conclusion:

MetaMask was integrated with a web-based DApp developed using HTML, CSS, and JavaScript to interact with the DrugSupplyChain smart contract deployed on the Ethereum Sepolia test network. Ethers.js was used to connect the frontend with the wallet and smart contract. The application supports drug registration, transfers between supply-chain participants, retrieval of drug details, and transaction verification. The experiment demonstrates how MetaMask enables users to authorize blockchain transactions and how Solidity smart contracts enforce supply-chain rules, providing transparency, traceability, and reliable record-keeping.